Sample Size Is Not Sampling Cost: A Validity Study for Configurable-System Sampling

Anonymous Author(s)
Submission to ICSE 2027 – Research Track

***Abstract*—Sampling-based performance prediction for configurable systems is commonly evaluated by accuracy and sample size. This treats the number of measured configurations as a proxy for acquisition cost. We test whether that proxy is valid. Across six evidence families–two replications of published configurable-system sampling settings and four extensions covering performance–energy trade-offs, workload sensitivity, hyperparameter-learning curves, and Linux kernel releases–we replace count-based effort with measured training cost: the sum of the measured costs of the configurations selected for training.**

Sample size is positively correlated with measured cost, so the proxy is not arbitrary. However, it is often not calibrated as a budget. Same-size samples can spend very different measured budgets; larger samples can be cheaper than smaller ones; and replacing count by measured cost changes the recommended strategy–budget combination in $538/1{,}666$ decision contexts. The effect is strongest under heavy-tailed costs, target–cost asymmetry, workload shifts, and samplers that seek diverse or uncertain regions. The result is a construct-validity warning: sample size is useful for ordering effort, but cost-effectiveness claims require measured-cost validation.

***Index Terms*—configurable systems, performance prediction, sampling, measurement cost, empirical software engineering**

I. Introduction

Performance prediction for configurable software systems is usually framed as a sampling problem: measure a small set of configurations, train a model, and use it to predict the rest of the configuration space [1]–[5]. Because measurements are expensive, sampling studies compare strategies on accuracy and "effort". In this literature, effort almost always means the number of measured configurations. A strategy that measures 64 configurations is treated as twice as expensive as one that measures 32, and two strategies that both measure 64 configurations are treated as equally expensive.

That convention is useful, but it is also a hypothesis. It assumes that each configuration costs roughly the same to measure, or at least that sampling strategies do not systematically choose more expensive or cheaper regions of the configuration space. Configurable systems give us many reasons to doubt that assumption. In a video encoder, compression options can multiply encoding time; in an archiver, compression settings can change energy consumption by orders of magnitude; in a compiler, optimization passes can turn a short compilation into a long one. In such settings, sample size remains a count, but it may no longer be a budget.

The issue is not merely accounting. Figure 1 gives the smallest version of the phenomenon on a real subject from our study: lrzip in the performance–energy corpus. In this scenario, the goal is to predict energy, but the acquisition budget is the measured performance cost of the configurations selected for training. All strategies measure the same 16 configurations. Count-based effort therefore assigns them the same cost. Measured performance cost does not: pilot-cost sampling spends only 0.034% of the full measurement budget, while diversity sampling spends 0.567%. Their prediction errors are almost indistinguishable (51.3% vs. 52.4% MRE), yet their measured costs differ by $16.5\times$. The absolute error is high because this is a deliberately small sample, but the budget lesson is the point: sample size can hide an order-of-magnitude cost difference even when prediction quality is essentially unchanged. Guizzo *et al.* made the analogous point for mutation testing: using the number of mutants as a surrogate for execution cost can threaten empirical validity [6]. We ask whether the same kind of surrogate-cost threat exists for sampling-based performance prediction.

Figure 1 also captures the paper's decision problem. If effort is counted as configurations, the strategies are tied on cost and the comparison reduces to error. If effort is measured as acquisition cost, the comparison becomes budget-dependent: two strategies with nearly equal error can be separated by more than an order of magnitude in real cost. The rest of the paper asks whether this reversal is an isolated artifact or a broader threat to sampling-study conclusions.

The same proxy failure can be even stronger than a same-size cost gap. In Figure 2, a `pilotCost` sample with 256 lrzip configurations is both cheaper to acquire and slightly more accurate than a `diversity` sample with 128 configurations. Under a count budget, the former is simply "larger" and therefore more expensive; under measured cost, it is the cheaper and better choice. This is why our question is not only whether n correlates with cost in aggregate, but whether n is a valid cost axis for the strategy choices a paper recommends.

This paper's thesis is therefore deliberately narrow:

> *Sample size can order sampling effort, but treating it as the cost axis is unsafe until the measured acquisition cost has been checked. In many realistic scenarios, that check changes the strategy recommendation or the set of efficient strategy–budget combinations.*

This is not a blanket criticism of prior work. Sample size is practical, reproducible, and often correlated with actual cost. Our claim is narrower: when a paper makes a resource claim, the count proxy should be checked against the measured acquisition cost. A statement such as "strategy A is cheaper

Sampling principle. A strategy selects n configurations from the option space; the measured runtime of those configurations is the acquisition cost, and their energy labels train the predictor.

$$\mathcal{C} \xrightarrow[\text{strategy } s]{n=16} S_{s,n}, \qquad TrainCost = \sum_{c \in S_{s,n}} \text{runtime}(c)$$

Excerpt of selected rows.

Strategy	enc.	comp.	method	lev.	cores	runtime
pilotCost	off	off	none	1	1	10.3k
pilotCost	off	off	none	2	3	10.4k
pilotCost	on	off	none	7	1	13.7k
diversity	on	on	zpaq	9	1	1.66M
diversity	off	on	zpaq	9	1	1.61M
diversity	off	on	lzma	9	4	288k

Rows are real selected configurations from the $n = 16$ samples. enc. and comp. are selected/deselected options; runtime is the cost paid before fitting the energy predictor.

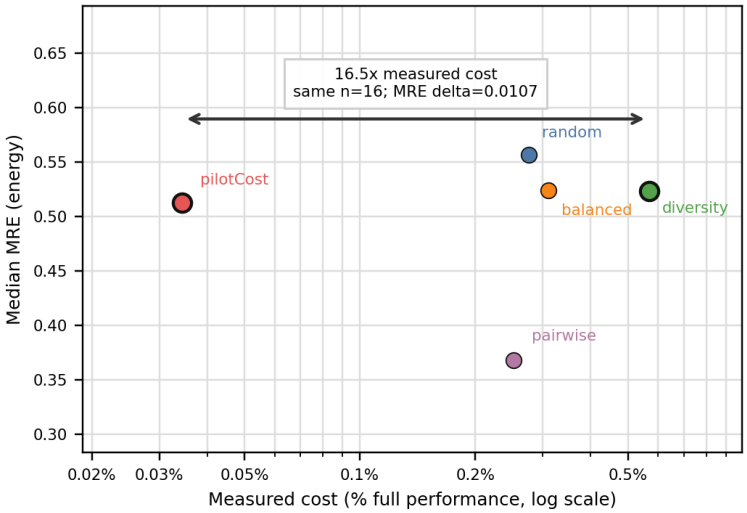


Fig. 1: Integrated running example on lrzip 0.631 compressing a 621 MB UIQ2 workload file (5,184 configurations, 12 option columns, runtime and energy measured over five repetitions). Left: the sampling principle and actual configuration excerpts show how two same-size samples can visit different option regions. Right: all strategies use $n = 16$, but measured performance cost separates nearly equal-error choices: pilot-cost and diversity differ by $16.5\times$ measured cost while MRE changes only from 51.3% to 52.4%.

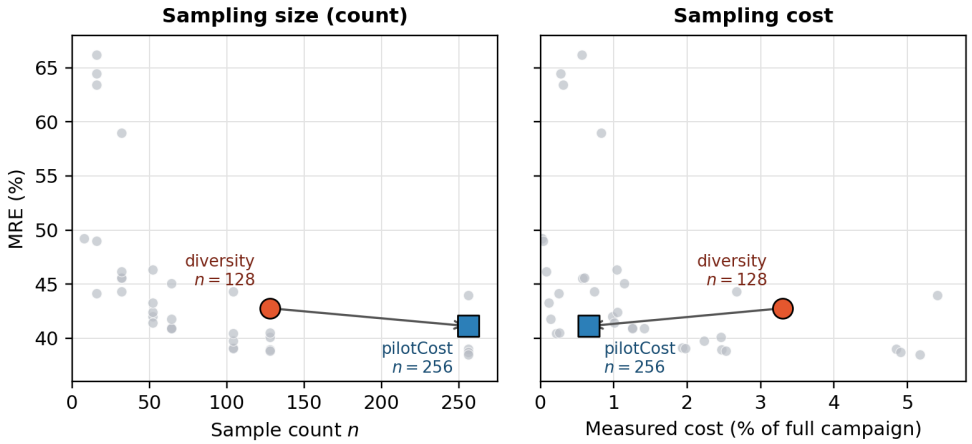


Fig. 2: A larger lrzip sample can be cheaper and more accurate under measured cost. The left panel uses sample count as the cost axis; the right panel uses measured acquisition cost for the same deployable strategy–budget combinations. The orange circle is `diversity` at $n = 128$ (MRE 42.77%, measured cost 3.297% of the full campaign). The blue square is `pilotCost` at $n = 256$ (MRE 41.11%, measured cost 0.658%). Grey points are the remaining deployable combinations in the same scenario.

than strategy B" is only meaningful after specifying whether "cheaper" means fewer configurations or less measured cost.

We test the size-as-cost abstraction by replacing count-based effort with measured acquisition cost[1]: the sum of the measured costs of the configurations a strategy selects. We study six complementary evidence blocks: a REPLICATION-ICSE19 revisit of Kaltenecker-style configurable-system traces, a REPLICATION-ICPE20 revisit of x264 over 17 inputs and two targets, a performance–energy corpus, a workload-impact corpus with 93 system–workload contexts, an LCBENCH external-validation corpus with 35 OpenML datasets and runtime learning curves, and seven Linux kernel releases with storage-bound cost. We ask three research questions:

1) to what extent sample size reflects measured acquisition cost;
2) whether replacing sample size by measured cost changes sampling-strategy conclusions; and
3) under which conditions sample size is a safe or unsafe cost proxy.

Agreement or disagreement across families is itself part of the result.

Contributions.

- We identify and formalize a construct-validity threat in

[1]Measured acquisition cost is the observed resource consumed to acquire a training measurement. For readability, we use *measured cost* in the remainder. *Empirical cost* is another possible terminology, but we avoid it to keep the resource interpretation explicit.

sampling-based performance prediction: sample count is often used as an effort budget, although configurations can have heterogeneous measurement costs.
- We test this threat across six evidence families, including two replications of published configurable-system sampling settings and four extensions that vary cost dimension, workload, dataset family, and scale.
- We quantify decision impact: replacing count by measured cost changes $538/1{,}666$ top strategy–budget decisions and removes many measured-cost-efficient choices from count-based Pareto fronts.
- We derive a lightweight reporting audit: report n, measured training cost, per-strategy budget ratio, and measured-cost Pareto fronts whenever making cost-effectiveness claims.

II. Background

A configurable system exposes options whose valid combinations form a configuration space C. For a fixed workload and non-functional property, each configuration $c \in C$ has a measured value y_c such as runtime, output size, or energy. Sampling-based performance prediction trains a regression model $\hat{y}$ from a measured subset $S \subset C$ and evaluates prediction error, typically mean relative error ($MRE = (1/|T|)\sum_{c\in T} |y_c - \hat{y}_c|/|y_c|$; lower is better), on held-out configurations T [3]–[5], [7].

Sampling strategies.: The configurable-systems literature contains several domain-agnostic sampling families. Random sampling draws configurations uniformly when enumeration is available. Solver-based sampling asks SAT/SMT solvers such as Sat4j or Z3 for valid configurations; randomized solver-based sampling perturbs solver order to increase diversity [8]. t-wise sampling covers combinations of option values [9]–[11]. Distance-based and diversified distance-based sampling select configurations to spread the sample in feature space [4]. When artifacts provide measured pools rather than feature-model synthesis handles, we use pool-based analogues: *balanced* sampling keeps option frequencies close to the pool, *coverage* greedily matches marginal option coverage, *diversity* spreads the sample by Hamming distance, *uncertainty* adds configurations with large predicted residuals, *pairwise* covers option-value pairs, and *pilot-cost* learns a small pilot cost model before choosing configurations predicted inexpensive. Cheap and expensive oracles select the lowest- and highest-cost measured configurations; they are diagnostic bounds, not deployable samplers.

How prior work counts cost.: Prior work has usually equated measurement effort with sample size. Sarkar *et al.* define a composite objective with training cost, testing cost, model cost, and error cost, but then collapse acquisition effort to a term proportional to n under unit per-configuration cost [2]. Westermann *et al.* discuss the number of measurement points needed for a target error [12]; Guo *et al.* vary several measurement budgets (N, $2N$, $3N$, and a larger reference budget M in their notation) [1]; Siegmund *et al.* report Pareto trade-offs between accuracy and number of measurements [3]; Kaltenecker *et al.* compare strategies at common sample sizes [4]. Siegmund *et al.* also report elapsed hours for some heuristics [13], but not as a test of whether n is a valid cost proxy. To our knowledge, the proxy itself has not been evaluated in this literature.

Context sensitivity.: Sampling conclusions already depend on workload and target property. Pereira *et al.* show that x264 strategy rankings change across 17 input videos and two properties [7]. We build on that observation, but ask a different question: not whether prediction accuracy is context-sensitive, but whether the validity of the cost proxy is context-sensitive. The two can come apart. A workload can leave rankings stable while changing the actual measurement budget, or keep the count and measured budgets close while changing the accuracy–cost frontier.

III. Cost Model

For a run r of strategy s at training size n, let $S_{r,s,n} \subseteq C$ be the selected sample. The usual cost model is the count of measured configurations:

$$SizeCost(s, n) = n. \tag{1}$$

We compare it with measured training acquisition cost:

$$TrainCost(r, s, n) = \sum_{c\in S_{r,s,n}} cost(c). \tag{2}$$

Here $cost(c)$ is the measured resource consumed to acquire the label for configuration c. In most of our study it is wall-clock measurement time; in the performance–energy family it can be energy. The Linux family is a resource-budget extension: the resource is uncompressed `vmlinux` size, so the question is whether the count proxy also fails when the paid resource is storage rather than time. The model does not require the cost dimension to equal the prediction target: one may predict output size while paying time, or predict runtime while paying energy.

To compare systems with different absolute costs, we normalize by the cost of measuring the whole available configuration space:

$$FullCost = \sum_{c\in C} cost(c), \tag{3}$$

$$TrainCostPct = \frac{TrainCost}{FullCost}, \qquad SizePct = \frac{n}{|C|}. \tag{4}$$

The count proxy has good *budget agreement* when the fraction of configurations measured matches the fraction of measured cost spent. We also call this *cost-proxy calibration*: the sample-size proxy is calibrated when a 10% sample spends about 10% of the measured budget. This is not statistical calibration of a prediction model; it is only a check that the cost proxy is on the same scale as the measured resource. We summarize it with a budget ratio:

$$Ratio(r, s, n) = \frac{TrainCostPct(r, s, n)}{SizePct(s, n)}. \tag{5}$$

$Ratio = 1$ means that a 10% sample consumes about 10% of the full measurement cost. $Ratio > 1$ means the strategy selected costlier-than-average configurations; $Ratio < 1$ means it selected cheaper-than-average ones. This is a per-run ratio; one-number summaries report the median ratio unless stated otherwise. The practical question is whether a same-size sample takes more or less of the real budget than the count suggests.

For decision-level comparisons, we use two standard views. First, we rank strategy–budget combinations, where each combination is one sampling strategy at one budget level, with a normalized objective

$$J_\lambda^{\text{size}} = \widetilde{MRE} + \lambda \widetilde{SizeCost}, \tag{6}$$

$$J_\lambda^{\text{meas}} = \widetilde{MRE} + \lambda \widetilde{TrainCost}, \tag{7}$$

where the cost weight $\lambda \in \{0.1, 0.25, 0.5, 1, 2, 4, 10\}$ spans accuracy-heavy to cost-heavy choices. Second, we compare Pareto fronts in the MRE–cost plane: a combination is efficient if no other combination has both lower error and lower cost. The main threat we test is whether the recommended strategy or the efficient set changes when $SizeCost$ is replaced by $TrainCost$.

The normalized objective is a decision lens, not a claim that every reader has the same utility function. An equivalent notation would write $\alpha\widetilde{MRE} + \beta\widetilde{Cost}$ with $\alpha, \beta \geq 0$; we fix the error coefficient to 1 and vary the relative cost weight $\lambda = \beta/\alpha$. Tildes are min–max normalization over the strategy–budget combinations in the same analysis context before MRE and cost are added; when all values are equal, the normalized column is set to zero. A small λ asks an accuracy-heavy question: which combination has the best error unless it is dramatically more expensive? A large λ asks a budget-heavy question: which combination preserves reasonable error while spending little measured cost?

This formulation also clarifies how the conventional fixed-n protocol fits our analysis. When all strategies are compared at the same sample size and the cost axis is sample count, the cost term is constant; the ranking collapses to MRE. That is the standard pure-accuracy reading of many sampling papers, and it remains valid for that question. The extra assumption appears when the same fixed-n comparison is read as a fixed-budget comparison. Our purpose is to test whether the same combination would still be recommended once the cost term is measured rather than counted.

This distinction is important for interpreting a "flip." If the count-cost winner is more accurate but also more expensive under measured cost, the result is a genuine trade-off: the reader may still prefer the extra accuracy. If a different combination is no less accurate and no more expensive under measured cost, the count-cost recommendation is dominated. Pareto fronts expose this second situation without choosing a particular λ: a front is the set of combinations that remain reasonable candidates for some accuracy–cost preference. When the measured-cost front contains combinations missing from the count-cost front, a size-only analysis has hidden choices a practitioner might legitimately prefer.

We count only training acquisition. Validation measurements are part of the experimental protocol and are the same across strategies; a sampling strategy controls which training configurations it asks us to measure.

IV. Research Questions and Study Design

The study asks one question: when is sample size a good enough proxy for measured acquisition cost, and when does that shortcut change the substantive conclusion? We organize the evaluation around three research questions.

- **RQ1. To what extent does sample size reflect measured acquisition cost?** We test whether same-size samples spend comparable measured budgets, whether larger samples reliably cost more, and whether a fixed fraction of configurations corresponds to the same fraction of measured acquisition cost.
- **RQ2. Does replacing sample size by measured acquisition cost change sampling-strategy conclusions?** We compare the top accuracy–cost choices and the Pareto-efficient choice sets under count-based cost and measured cost.
- **RQ3. Under which conditions is sample size a safe or unsafe cost proxy?** We analyze how the answer depends on cost heterogeneity, strategy bias, target–cost asymmetry, workload/input, release, and dataset family.

Operational tests.: The tests are deliberately simple because the construct is simple: if sample size is a good budget proxy, configurations should have similar acquisition costs, larger samples should spend more measured cost, and strategy recommendations should not depend on whether cost is counted or measured.

RQ1 first checks *cost heterogeneity*. We use the coefficient of variation (CV), the standard deviation divided by the mean, because it is unit-free: a CV of 0.20 means the standard deviation is 20% of the mean whether the resource is seconds, joules, or bytes; a CV near 1 means the spread is as large as the average cost. CV is sensitive to broad dispersion, so we pair it with p_{90}/p_{10}, the ratio between the 90th and 10th percentile costs. This second statistic is robust and readable: $p_{90}/p_{10} = 1.5$ means a costly but non-extreme configuration costs 50% more than a cheap but non-extreme one; a value near 47 means the upper tail is about $47\times$ the lower tail. Fixed-n strategy-effect tests then ask the operational question: when two strategies both choose, say, 64 configurations, do they nevertheless visit systematically cheaper or costlier regions?

RQ1 also checks *budget calibration*. Spearman ρ between n and $TrainCost$ asks only whether larger samples tend to cost more. The ratio $Ratio$ from Equation (5) asks the stricter budget question. If a system has $1{,}000$ configurations and a sample measures 100, then $SizePct = 0.10$. If those 100 configurations spend 8% of the full measured campaign, then $Ratio = 0.08/0.10 = 0.8$: the sample is cheaper than its count suggests. If it spends 15%, then $Ratio = 1.5$: the sample is costlier than its count suggests.

RQ2 uses decision tests. A top-1 flip means that the strategy–budget combination recommended by the same accuracy–cost objective changes after replacing count cost with measured cost. Pareto-front Jaccard overlap measures the broader choice set: Jaccard $= 1$ means the count-cost and measured-cost efficient sets are identical; lower values mean that some trade-offs visible under one cost model disappear under the other. RQ3 interprets the same statistics across contexts. We report p-values where they are meaningful, but always pair them with practical thresholds: $\widetilde{\text{CV}} \geq 0.20$ or $p_{90}/p_{10} \geq 1.5$ for visible cost spread, $|\widetilde{Ratio} - 1| \geq 0.05$ for budget mismatch, and Jaccard < 1 for a changed efficient choice set. Full per-scope tests, p-values, and effect sizes are in the supplementary artifact.

A. Evidence Families

We keep six evidence families separate because they stress different validity conditions. Pooling them would give a larger number but a weaker explanation of when the sample-size proxy is reliable. The two replication families keep the link with published configurable-system sampling studies: one revisits seven Kaltenecker-style systems, and the other revisits x264 over 17 inputs and two measured properties. The four extension families then change one condition at a time: the cost dimension (performance versus energy), the workload or input, an external hyperparameter-optimization corpus, and a large Linux storage-cost setting.

The REPLICATION-ICSE19 family is the most conservative test. It reuses the published strategy panel–random, solver-based, randomized solver-based, t-wise, distance-based, and diversified distance-based sampling–and changes only the cost axis from sample count to measured performance cost. If a count-based conclusion changes there, the change is not caused by inventing a new sampler or a new benchmark. The REPLICATION-ICPE20 family uses the same logic on x264, but adds context: the same $1{,}152$ configurations are observed on 17 videos and two properties, so we can ask whether the cost proxy is safer when target and acquisition cost are the same than when they differ.

The extension families deliberately broaden the construct. ENERGY asks whether the issue is only about runtime or also appears when performance and energy can each be target or cost. WORKLOAD keeps systems fixed but changes workloads, which tests whether a conclusion about the cost proxy is a property of the system or of the system–workload pair. LCBENCH moves outside configurable-software performance tables and checks whether real runtime learning curves in hyperparameter optimization show the same count-versus-cost gap. LINUX stresses scale and strategy behavior: random, balanced, coverage, uncertainty, and diversity sample from large randconfig pools, and acquisition cost is the uncompressed kernel image size rather than time.

Table I gives the factual dataset scale. The table is descriptive rather than interpretive: the result sections explain what these data imply for the three RQs.

B. Protocol

For each available tuple of system, workload/input, target, strategy, training size, and run, we replay or generate the training sample, compute $TrainCost$, train the predictor on configuration options only, and evaluate MRE on held-out configurations. Stochastic strategies use repeated seeds when available, typically 100 runs. We exclude measured performance, energy, size, and runtime columns from the feature matrix to avoid label leakage, especially when the target and acquisition cost are different measured quantities.

Strategy panels and diagnostic bounds.: The main panels use deployable strategies from the source studies or from our generated-sampling protocol. Some generated families also include `cheapOracle` and `expensiveOracle`; these are diagnostic bounds that assume access to true cost labels before sampling. We label them as oracles and never interpret them as deployable recommendations.

Artifact.: The anonymized artifact records selected configurations, strategy identifiers, seeds, measured targets, measured acquisition costs, scripts for recomputing $TrainCost$, $Ratio$, top-choice flips, and Pareto fronts, and the provenance of all tables and figures.

V. Results

The empirical answer is not "sample size is useless." It is sharper: *sample size reliably orders effort, but it often hides the budget and can change decisions*. We therefore report the families separately. The REPLICATION-ICSE19 and REPLICATION-ICPE20 families ask whether published count-based conclusions survive a measured-cost axis. The ENERGY, WORKLOAD, LCBENCH, and LINUX families test whether the same phenomenon survives changes in cost dimension, workload, dataset family, and configuration-space scale.

A. RQ1: Sample Size Orders Cost, But Does Not Budget It

Sample size is a useful ordering signal. In every family where we can test it, larger samples are positively rank-correlated with measured acquisition cost. This matters: count-based plots are not arbitrary, and n remains a useful common language for comparing sampling campaigns. In the REPLICATION-ICSE19 revisit, Spearman correlation between sample size and measured cost is high ($\rho = 0.879$–0.943). The same ordering signal appears in the generated families as well: ENERGY remains positive even in its hardest cases ($\rho = 0.710$–0.996), and LINUX is near monotone across the seven releases.

The failure is budget calibration. At fixed n, strategies can visit very different cost regions. In the running example, five lrzip strategies all use $n = 16$, yet pilot-cost sampling spends 0.034% of the full measured performance budget while diversity spends 0.567%, a $16.5\times$ gap for nearly identical MRE. Across the broader evidence, spread ranges from benign cases such as LLVM ($p_{90}/p_{10} = 1.21$) to heavy-tailed cases such as lrzip and BerkeleyDBC ($p_{90}/p_{10} \approx 47$), jump3r performance (CV 7.104), Linux storage cost (CV 0.99–1.43),

TABLE I: Dataset characteristics. Configurations are measured configurations available per context unless noted otherwise; NFPs are the measured non-functional properties used as targets or acquisition costs.

Family	Scope	Configurations	Options/features	NFPs used	Inputs/workloads
REPLICATION-ICSE19	Seven configurable systems from Kaltenecker *et al.* [4].	432–68,640 valid configurations per system.	11–54 options.	One performance-like target per system; the same measured value is charged as cost.	One context per system.
REPLICATION-ICPE20	x264 ICPE 2020 [7].	1,152 valid configurations.	16 binary options.	Encoding time and output size, combined into four cost–target scenarios.	17 input videos.
ENERGY	Seven systems from the performance–energy corpus.	180–6,840 measured configurations per system.	System-specific option vectors from the source artifacts.	Performance and energy, combined into four scenarios per system.	One workload per selected system.
WORKLOAD	Nine systems from the workload-impact ICSE 2023corpus.	191–6,764 configurations per system; 25,258 measurements overall.	System-specific option vectors from the source artifacts.	Wall-clock time, or oriented throughput for H2.	6–13 workloads per system, 93 contexts total.
LCBENCH	35 OpenML datasets from LCBench.	2,000 hyperparameter configurations per dataset.	7 hyperparameters.	Validation accuracy as target, total runtime as cost.	35 dataset contexts.
LINUX	Seven TuxKConfig Linux releases.	19,952–92,562 valid configurations per release.	494–497 retained kernel options.	Uncompressed `vmlinux` size as target and storage-cost proxy.	Seven releases, v4.13–v5.08.

TABLE II: Measured-cost audit for sampling claims. The checks make explicit which construct is being used for cost, whether sample count is calibrated to that construct, and whether conclusions survive the replacement.

Check	What to report	Evidence in this paper	Failure signal
Cost construct	State the resource paid to acquire labels, separately from the prediction target.	x264 and ENERGY cross target/cost dimensions; LINUX uses storage-bound resource cost.	Target and acquisition cost are silently conflated.
Cost spread	Report per-configuration heterogeneity, e.g., CV or p_{90}/p_{10}.	CV reaches 7.104 in ENERGY; ratios approach $47\times$ in replications.	Equal-price assumption is implausible.
Budget calibration	Compare $n/\lvert C\rvert$ with $TrainCost/FullCost$ by strategy and budget.	$Ratio$ ranges from 0.168 to near 1; Linux has aggregate calibration but strategy bias.	Count is an ordering variable, not a budget.
Decision impact	Recompute top choices and measured-cost Pareto fronts.	$538/1{,}666$ top decisions change; count fronts miss about 52% of measured-cost-efficient choices.	Cost-aware conclusions change.
Artifact trace	Save selected configurations, strategy/seed, target, and measured acquisition cost.	Released artifacts recompute $TrainCost$, $Ratio$, flips, and fronts.	Reviewers cannot audit the construct.

and LCBENCH runtime (CV 0.595–2.247). The budget ratio tells the same story: a nominal 10% sample can spend close to 10% of the measured budget, but BerkeleyDBC has median $Ratio = 0.797$, ENERGY reaches $Ratio = 0.168$, workload contexts span 0.661–1.059, and LCBENCH spans 0.669–1.012.

The x264 revisit explains why aggregate calibration is not enough. Its time budgets look well calibrated when averaged over inputs, yet strategies are not interchangeable: Henard-style samples tend to be cheaper, whereas t-wise samples tend to be costlier. Thus a family can pass the coarse "does n order cost?" check and still fail the same-size, strategy-specific budget check that matters for recommendations.

This distinction matters because the three RQ1 checks can disagree. Rank correlation asks only whether larger samples tend to cost more; it supports the traditional use of n as an ordering variable. Budget calibration asks whether the count has the right scale; it fails when $Ratio$ is far from 1. Strategy bias asks whether two samples with the same n can spend different budgets; it is the mechanism behind most practical failures. Linux illustrates the split: random-like strategies stay close to the count budget, while diversity costs 1.24–$1.52\times$ its count share at evaluated small-budget levels and uncertainty can be cheaper or costlier depending on release and sample size. Thus an aggregate plot can look safe while a specific strategy is not.

Figure 2 shows the sharpest local failure. Under the count view, `pilotCost` at $n = 256$ is simply the larger sample and `diversity` at $n = 128$ is the smaller one. Under measured cost, the ordering reverses: `pilotCost` spends only 0.658% of the full measured campaign, whereas `diversity` spends 3.297%. The larger sample is therefore $5.0\times$ cheaper and slightly more accurate (MRE 41.11% vs. 42.77%). Thus sample size usually orders cost in aggregate, but individual strategy–budget choices can violate the ordering implied by the count proxy.

Answer to RQ1

Sample size is a useful ordering variable, but it is not a calibrated budget. Across all families, larger samples usually cost more; nevertheless, same-size samples can differ by orders of magnitude in measured cost, and a larger sample can be cheaper than a smaller one when the strategy selects cheaper configurations.

B. RQ2: Measured Cost Changes Strategy Assessment

Replacing sample size by measured cost changes conclusions in two ways. The stronger change is a different top strategy–budget combination under the same accuracy–cost objective. The broader change is a different Pareto-efficient choice set: even when the top choice is stable, count cost can hide cheaper same-size alternatives or keep costlier combinations that are no longer efficient.

Across the primary artifacts, the top strategy–budget combination changes in $538/1{,}666$ context–weight decisions (32%) when measured cost replaces count cost. In $293/538$ changed decisions (54%), the count winner is more accurate but costlier

to acquire; this is a real trade-off, not an automatic refutation. The stricter same-budget failure occurs when the count winner is not on the measured-cost Pareto frontier: another combination is no less accurate and no more expensive. That occurs in $57/538$ changed decisions (11%). Count-cost frontiers also miss about 52% of the combinations that are efficient under measured cost. Table II summarizes the checks needed to make those changes auditable.

Here a *recommendation* is concrete: for one system, input or workload, target, and cost weight, it is the strategy and budget level the objective would tell a practitioner to use. A changed recommendation therefore means that the paper would name a different sampling strategy or a different sampling size once the measured acquisition budget is charged. A *choice set* is broader: it contains all non-dominated strategy–budget combinations that remain plausible for some preference between accuracy and cost. A stable top recommendation can still hide a changed choice set, and that matters when users have different budgets or error tolerances.

Both effects appear repeatedly. In the REPLICATION-ICSE19 revisit, LLVM, Polly, Dune, and HIPAcc validate the top count-based recommendation, but lrzip changes in $5/7$ cost-weight settings and frontiers change on $5/7$ systems. The x264 revisit is stable in the symmetric time/time case, yet the time-cost/size-target scenario changes $21/119$ input–weight decisions and has median Pareto Jaccard 0.375. This is a practical case: the target of interest is not always the bottleneck acquisition resource. Beyond the replications, top combinations change in $24/28$ performance–energy scenarios, $72/93$ workload contexts, $33/35$ LCBENCH datasets, and every Linux release. Pareto fronts differ in all 28 performance–energy scenarios, $71/93$ workload contexts, $32/35$ LCBENCH datasets, and all seven Linux releases.

Figure 3 illustrates a common source of confusion. The property we want to predict and the resource we pay to acquire a training label need not be the same. For energy prediction, the label is energy, but obtaining that label requires running the configuration; execution time is therefore a natural acquisition cost. Charging energy as the cost simply because energy is the target would answer a different question. In Figure 3, the acquisition resource is held fixed as execution time; only the target changes. The cost-model conclusion changes with that target: the time/time case validates the count recommendation, while the time/energy case does not. The cost model is therefore part of the experimental question, not a synonym for the response variable.

The changed conclusion is not always "the count winner was worse." Often the count-based winner is more accurate but costlier; then the real question is whether that accuracy gain is worth the measured budget. In other cases the failure is stricter: a same-size or larger alternative is both cheaper and no less accurate under measured cost, as in Figure 2. The scientific recommendation should therefore state which cost model it uses. "Best at sample size n" and "best at measured acquisition budget" are not the same claim.

This resolves an apparent tension in the examples. If the goal is pure MRE at a fixed sample size, the most accurate count-based strategy is the right answer: the study has intentionally held n constant and abstracted away acquisition cost. If the goal is cost-aware acquisition, the question changes to whether the same MRE could be reached with less measured cost, or whether the same measured budget could buy lower MRE through a different strategy–budget combination. RQ2 therefore does not penalize accuracy; it asks whether the accuracy improvement is still persuasive after charging the resource actually spent to obtain the training labels.

Answer to RQ2

Replacing sample size by measured cost changes both top recommendations and the efficient choice set. Across $1{,}666$ context–weight decisions, the top strategy–budget combination changes in 538 cases (32%). Some changes are genuine accuracy–cost trade-offs; the strict failures are the 57 changed cases where the count-cost winner is dominated under measured cost.

C. RQ3: Validity Conditions

The count proxy is most defensible when per-configuration costs are tight, when the target and acquisition resource are the same, and when the strategy does not bias itself toward costly regions. It is most fragile when costs are heavy-tailed, when target and cost differ, when workload or dataset changes the cost distribution, or when the strategy explicitly seeks diversity or uncertain regions.

The validated cases are useful because they bound the claim. LLVM, Polly, Dune, HIPAcc, PostgreSQL, and x264 time/time show that count-based conclusions can survive when cost spread is low or strategies sample similar regions. The warning cases are different: 7Z and BerkeleyDBC have changed frontiers but few top flips, so sample size is insufficient for reporting the efficient choice set even if the headline winner mostly survives. The changed-assessment cases combine cost spread with decision movement: lrzip flips in most cost weights, x264 changes most under time-cost/size-target asymmetry, LCBench changes on most datasets, and Linux changes on every release. The validity condition is therefore not "configurable systems are unsafe"; it is "the safety of the count proxy is itself a property to measure."

The mechanisms differ in severity. A same-size cost gap is the basic warning: the count axis has lost budget information, but the preferred sampler may still be unchanged. Similar-MRE/different-cost cases suggest wasted acquisition budget without an accuracy payoff. Larger-sample-but-cheaper cases are stronger still, because they contradict the monotone intuition that more configurations necessarily means more cost. The final pattern, where conventional same-budget samplers separate under measured cost, is closest to prior sampling papers: even familiar fixed-n comparisons can change when configurations have heterogeneous acquisition costs.

Cross-RQ synthesis.: The families fall into three practical regimes. In the *validated-transfer* regime, count-based conclusions survive the measured-cost check. LLVM, Polly, Dune, HIPAcc, PostgreSQL, and x264 time/time are the main

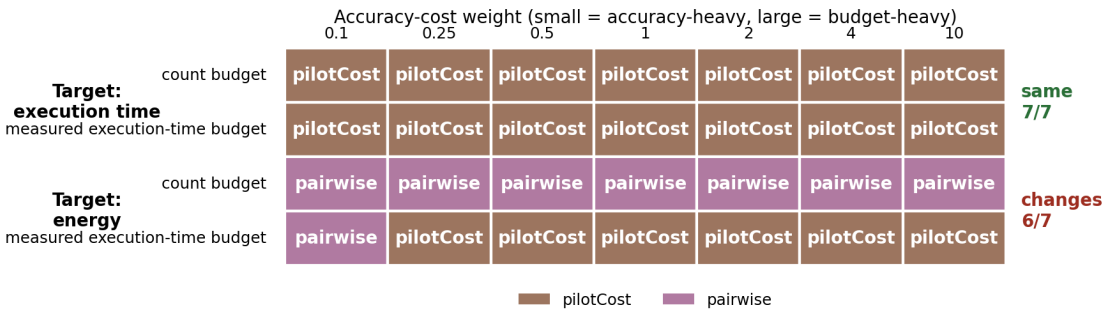


Fig. 3: Target–cost asymmetry in ENERGY on exastencils, using only ordinary strategies. Both blocks charge the same acquisition resource: measured execution time. When the target is also execution time, count cost and measured execution-time cost recommend the same strategy for all seven accuracy–cost weights. When the target is energy, while acquisition still costs execution time, the recommendation changes for six of seven weights. This is why measured cost should be the resource paid to obtain labels, not automatically the property being predicted.

examples: costs are tight enough, or the strategies are cost-blind enough, that replacing count by measured cost does not change the top recommendation. In the *warning* regime, the top recommendation mostly survives but the efficient choice set changes. 7Z and BerkeleyDBC fit this pattern: a reader who only wants the single winner may reach the same conclusion, but a reader comparing trade-offs would miss measured-cost-efficient alternatives. In the *changed-assessment* regime, measured cost changes the recommendation itself. lrzip, the asymmetric x264 time-cost/size-target scenario, heavy-tailed performance–energy systems, many workload contexts, most LCBENCH datasets, and every Linux release fall here. The scientific message is not that one sampler dominates in all settings; it is that the validity of the count proxy is an empirical property of the dataset, workload, cost dimension, and strategy panel.

Answer to RQ3

Sample size is safest as a cost proxy when per-configuration costs are tight, the target and bottleneck resource coincide, and strategies are cost-blind. It is unsafe when costs are heavy-tailed, target and acquisition cost differ, workload or dataset changes the cost distribution, or the strategy deliberately searches unusual, diverse, or uncertain regions.

VI. Discussion

How to read prior count-based results.: The results do not make prior sampling studies "wrong." A study that compares strategies at the same n and claims only prediction accuracy is answering a fixed-size accuracy question. The problem starts when that result is read as an acquisition-cost claim, or when a larger n is assumed to mean a larger real budget. Then sample count becomes an untested construct. The safest reading is conditional: count-based conclusions transfer when costs are homogeneous enough or strategies are cost-blind enough; they need measured-cost restatement when strategies visit systematically cheaper or costlier regions.

What changes in future papers.: The reporting change is lightweight. A sampling paper should state the paid resource, report sample size n, report measured acquisition cost $TrainCost$, and show per-strategy $Ratio$ at the nominal budgets it uses. If costs are homogeneous and ratios stay near 1, count is a defensible budget proxy. If costs are heavy-tailed, target and cost differ, or strategies enter costly regions, accuracy–cost conclusions should be stated under measured cost. The artifact fields needed for this audit are simple: selected configuration, strategy, seed, measured target, and measured acquisition cost.

Does this imply a new sampler?: The evidence creates room for cost-aware sampling, but it does not identify a universal replacement strategy. Diagnostic cheap/expensive oracles bound what would be possible if costs were known before sampling; they are not deployable baselines. `pilotCost` is closer to a practical idea–learn cheap regions from a small pilot–but it can fail if cheap regions are uninformative for the target. The contribution here is the prior step: determine whether the standard count axis is valid for the study at hand, and reveal when a cost-aware sampler would have room to improve the trade-off.

When measured cost matters most.: Large cost heterogeneity creates room for failure, but strategy bias and context decide whether the failure changes a recommendation. BerkeleyDBC and 7Z are warning cases: frontiers change even when top choices mostly survive. lrzip, brotli, jump3r, LCBench, and Linux show the stronger case: cost spread combines with strategy panels that select different cost regions. x264 adds a separate lesson: a property can be cheap to predict but expensive to acquire, or vice versa, so target–cost asymmetry can matter even when aggregate calibration looks benign.

VII. Threats to Validity

Cost construct.: We define acquisition cost as the sum of measured per-configuration costs in existing artifacts. This captures the benchmarked resource, not every production overhead such as orchestration, retries, or warm-up. The relative questions we ask–whether same-size samples choose cheaper or costlier configurations–are less sensitive to machine-specific absolute times.

Protocol heterogeneity.: The evidence families use different strategy panels, learners, workloads, and cost dimensions, so we report them separately rather than pooling one effect

size. RQ1 is learner-independent; RQ2 and RQ3 depend on MRE and could move with another learner or error metric. We keep original learners for the two replication families and report learner-sensitivity artifacts for stronger learners and newly framed families.

Learner choice.: A better learner can improve MRE and can change the winning strategy–budget combination, so it is a threat to any claim about a specific top-ranked sampler. It does not, however, define acquisition cost: RQ1 is computed before fitting any model, from the selected configurations and their measured costs. The Linux HGBT sensitivity run is the key sanity check. It substantially improves accuracy, yet measured cost still changes rankings and Pareto fronts on every release. Better predictors can change which combination wins; they do not make sample size a sufficient cost model.

External validity and multiple tests.: The systems cover compilers, encoders, databases, archivers, energy measurements, workloads, machine-learning runtime, and Linux releases, but they remain benchmark artifacts. Practitioners should transfer the check, not the exact thresholds: inspect cost spread and per-strategy budget ratios on the target campaign. Because we run many per-scope tests, main claims rely on repeated patterns across families; the supplementary artifact records the per-scope statistics, p-values, and effect sizes.

VIII. Related Work

Sampling configurable systems.: Sampling-based performance prediction has studied random, solver-based, coverage-based, distance-based, progressive, and transfer-learning strategies [1]–[5], [14]. Most papers evaluate effort as sample size. Sarkar *et al.*'s cost-efficient model is the closest configurable-system precedent [2], but its acquisition term is still proportional to the number of measured configurations. Our contribution is not a new sampler; it is a validity study of that effort axis.

Counting effort versus measuring acquisition cost.: Across tuning, transfer, and prediction, "effort," "efficiency," and "speedup" are often expressed as counts–measurements, evaluations, queries, or target-environment samples–because counts are simple to compare and reuse [15]–[20]. Counts sound like budgets only if one count unit stands for a comparable amount of acquisition cost. Otherwise, a cost-effective result can be an accuracy result wearing a cost label. Some tuning work already grounds counts in measured wall-clock budgets [21], [22]; our results show why per-configuration and per-strategy calibration is still needed.

Surrogate and heterogeneous costs.: Replacing a real resource with an easy-to-count surrogate is a recurring validity concern. In mutation testing, for example, the number of mutants can mislead relative to execution time [6]. We ask the analogous question for configuration sampling. The mechanism is specific: sample size is usually correlated with measured cost, yet strategy-specific budget agreement and decision-level frontiers can change because some configurations are slow, energy-intensive, storage-heavy, or workload-dependent.

Context, learners, and optimization.: Prior work shows that sampling conclusions can vary across inputs, workloads, and target properties [7], [23]; we use these dimensions to test cost-proxy validity. Learner choice is a separate modeling issue: we keep literature-aligned learners for replications and use stronger sklearn baselines as robustness checks or primary learners for newly framed families [24]–[28]. Active learning and multi-objective optimization offer candidate solutions, but they inherit the same proxy question through their cost term [2], [3], [29]. We therefore treat measured cost as a prerequisite for accuracy–cost claims, not as a new optimizer.

Other resource axes.: A distinct strand measures model-building or adaptation cost rather than label acquisition: encoding-scheme comparisons report training time [30], online drift adaptation reports measured overhead [31], and the DCPL survey maps deep-learning costs [32]. These axes are complementary. Our analysis counts the resource spent to obtain training labels; extending the same audit to fitting time, energy, bandwidth, or human labeling cost is natural future work.

IX. Conclusion

Sampling cost matters. For configurable-system performance prediction, the number of measured configurations is a useful summary, but it is not a resource budget unless it has been checked against measured acquisition cost. Across the systems we studied, n almost always orders measured cost, yet it can misstate the budget, hide strategy-specific bias, and change which strategy or strategy–budget combination should be recommended.

The decision impact is concrete. Across $1{,}666$ context–weight decisions, the top strategy–budget combination changes in 538 cases (32%). Many changes are legitimate accuracy–cost trade-offs, but 57 are stricter same-budget failures where the count-based winner is not on the measured-cost Pareto frontier. Count-based frontiers also miss about 52% of the strategy–budget combinations that are efficient under measured cost.

The main result is a validity warning, not a blanket rejection of prior work. Count-based effort is defensible for homogeneous systems and cost-blind samplers, and our data identifies several such cases. It is unsafe for heavy-tailed costs, target–cost asymmetry, workload-sensitive cost distributions, non-pure software-system dataset shifts, and diversity or active-learning strategies that drift toward costly regions. These conditions are common, measurable, and severe enough to change winners and Pareto fronts; future studies should not leave them implicit.

The practical fix is lightweight: report sample size, report measured training cost, and state how well the two budgets agree. Doing so turns "effort" from an unspoken convention into a verifiable part of the experimental design. The new LCBENCH family shows that this warning is not confined to configurable software performance corpora: the same count-vs-cost gap survives on runtime learning-curve benchmarks outside pure software-system studies.

ACKNOWLEDGMENTS

Anonymized for review.

REFERENCES

[1] J. Guo, K. Czarnecki, S. Apel, N. Siegmund, and A. Wasowski, "Variability-aware performance prediction: A statistical learning approach," in *Automated Software Engineering (ASE)*. IEEE, 2013, pp. 301–311.

[2] A. Sarkar, J. Guo, N. Siegmund, S. Apel, and K. Czarnecki, "Cost-efficient sampling for performance prediction of configurable systems (t)," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2015, pp. 342–352.

[3] N. Siegmund, A. Grebhahn, S. Apel, and C. Kastner, "Performance-influence models for highly configurable systems," in *10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE, 2015, pp. 284–294.

[4] C. Kaltenecker, A. Grebhahn, N. Siegmund, J. Guo, and S. Apel, "Distance-based sampling of software configuration spaces," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2019.

[5] J. A. Pereira, H. Martin, M. Acher, J.-M. Jézéquel, G. Botterweck, and A. Ventresque, "Learning software configuration spaces: A systematic literature review," 2019.

[6] G. Guizzo, F. Sarro, and M. Harman, "Cost measures matter for mutation testing study validity," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 1127–1139. [Online]. Available: https://doi.org/10.1145/3368089.3409742

[7] J. Alves Pereira, M. Acher, H. Martin, and J.-M. Jézéquel, "Sampling effect on performance prediction of configurable systems: A case study," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 277–288. [Online]. Available: https://doi.org/10.1145/3358960.3379137

[8] C. Henard, M. Papadakis, G. Perrouin, J. Klein, P. Heymans, and Y. L. Traon, "Bypassing the combinatorial explosion: Using similarity to generate and prioritize t-wise test configurations for software product lines," *IEEE Trans. Software Eng.*, 2014.

[9] D. Kuhn, D. Wallace, and A. Gallo, "Software fault interactions and implications for software testing," *IEEE Transactions on Software Engineering*, vol. 30, no. 6, pp. 418–421, jun 2004.

[10] M. B. Cohen, M. B. Dwyer, and Shi, Jiangfan, "Constructing Interaction Test Suites for Highly-Configurable Systems in the Presence of Constraints: A Greedy Approach," *IEEE TSE*, vol. 34, no. 5, pp. 633–650, 2008.

[11] M. F. Johansen, Ø. Haugen, and F. Fleurey, "An Algorithm for Generating t-Wise Covering Arrays from Large Feature Models," in *Proceedings of the 16th International Software Product Line Conference on - SPLC '12 -volume 1*, vol. 1. ACM, 2012, p. 46.

[12] D. Westermann, J. Happe, R. Krebs, and R. Farahbod, "Automated inference of goal-oriented performance prediction functions," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2012, pp. 190–199.

[13] N. Siegmund, S. S. Kolesnikov, C. Kästner, S. Apel, D. S. Batory, M. Rosenmüller, and G. Saake, "Predicting performance via automated feature-interaction detection," in *International Conference on Software Engineering (ICSE)*, 2012, pp. 167–177.

[14] V. Nair, T. Menzies, N. Siegmund, and S. Apel, "Faster discovery of faster system configurations with spectral learning," *Automated Software Engineering*, pp. 1–31, 2018.

[15] V. Nair, Z. Yu, T. Menzies, N. Siegmund, and S. Apel, "Finding faster configurations using FLASH," *IEEE Transactions on Software Engineering*, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2018.2870895

[16] P. Chen, T. Chen, and M. Li, "Mmo: Meta multi-objectivization for software configuration tuning," *IEEE Transactions on Software Engineering*, 2024, preprint at arXiv:2112.07303. [Online]. Available: https://doi.org/10.1109/TSE.2024.3388910

[17] T. Chen and M. Li, "Adapting multi-objectivized software configuration tuning," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2024. [Online]. Available: https://doi.org/10.1145/3643751

[18] Y. Ye, T. Chen, and M. Li, "Distilled lifelong self-adaptation for configurable systems," *arXiv preprint arXiv:2501.00840*, 2025, accepted at ICSE 2025. [Online]. Available: https://arxiv.org/abs/2501.00840

[19] J. Gong and T. Chen, "Predicting software performance with divide-and-learn," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2023. [Online]. Available: https://doi.org/10.1145/3611643.3616334

[20] ——, "Predicting configuration performance in multiple environments with sequential meta-learning," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2024, preprint at arXiv:2402.03183. [Online]. Available: https://doi.org/10.1145/3643743

[21] T. Chen and M. Li, "Multi-objectivizing software configuration tuning (for a single performance concern)," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021. [Online]. Available: https://doi.org/10.1145/3468264.3468555

[22] ——, "Do performance aspirations matter for guiding software configuration tuning?" *ACM Transactions on Software Engineering and Methodology*, 2023. [Online]. Available: https://doi.org/10.1145/3571853

[23] Y. Ye, H. Liang, C. Jiang, M. Li, and T. Chen, "Revealing domain-spatiality patterns for configuration tuning: Domain knowledge meets fitness landscapes," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://doi.org/10.1145/3803859

[24] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[25] P. Valov, J. Guo, and K. Czarnecki, "Empirical comparison of regression methods for variability-aware performance prediction," in *SPLC'15*, 2015.

[26] B. Amand, M. Cordy, P. Heymans, M. Acher, P. Temple, and J.-M. Jézéquel, "Towards learning-aided configuration in 3d printing: Feasibility study and application to defect prediction," in *Proceedings of the 13th International Workshop on Variability Modelling of Software-Intensive Systems*. ACM, 2019, p. 7.

[27] L. Bao, X. Liu, Z. Xu, and B. Fang, "Autoconfig: Automatic configuration tuning for distributed message systems," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2018, pp. 29–40.

[28] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[29] M. Li, T. Chen, and X. Yao, "How to evaluate solutions in pareto-based search-based software engineering? a critical review and methodological guidance," *IEEE Transactions on Software Engineering*, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2020.3036108

[30] J. Gong and T. Chen, "Does configuration encoding matter in learning software performance? an empirical study on encoding schemes," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022. [Online]. Available: https://doi.org/10.1145/3524842.3528431

[31] Z. Xiang, J. Gong, and T. Chen, "Dually hierarchical drift adaptation for online configuration performance learning," in *Proceedings of the International Conference on Software Engineering*, 2026. [Online]. Available: https://doi.org/10.1145/3744916.3764542

[32] J. Gong and T. Chen, "Deep configuration performance learning: A systematic survey and taxonomy," *ACM Transactions on Software Engineering and Methodology*, 2024, preprint at arXiv:2403.03322. [Online]. Available: https://doi.org/10.1145/3702986